Lecture 6: Direct Preference Optimization

RLHFBOOK.COM

Nathan Lambert

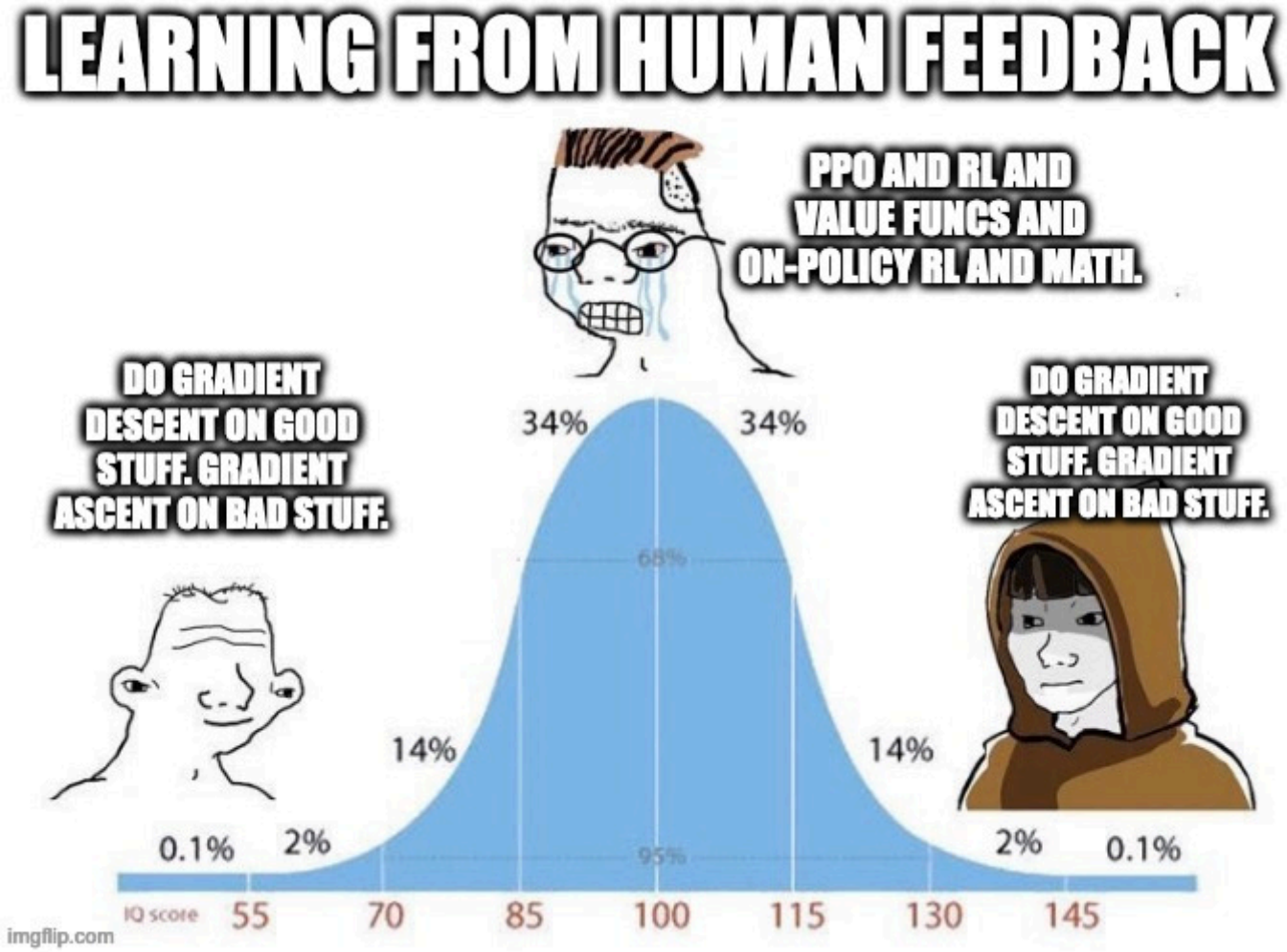
Post-training course. Chapter 8 on Direct Alignment Algorithms.

Why DPO was – and is – such a big deal

The Direct Preference Optimization paper (Rafailov et al., 2023) was a breakthrough in the accessibility of RLHF and post-training research. Some context:

- DPO paper was released in May of 2023, when many groups were struggling to get open-source replications of RLHF pipelines going. Most "aligned" models were just SFT.
- It took until about the fall of 2023 for people to figure out the right data and settings for DPO to work
- Zephyr-Beta, Tülu 2, and other models that fall opened the floodgates of post-training research. Iterating on DPO methods in 2024 felt as exciting as RL methods in 2025 or tool-use methods in 2026.
- **Terminology note:** I call the class of algorithms that operate like DPO on static preference data Direct Alignment Algorithms (DAAs). The name didn't stick much, but we'll see there are a ton of variants!

The DPO debate (DPO v RL)



When DPO was released it sparked a fierce debate about how best to do RLHF and preference learning. Meme credit: Tom Goldstein.

This lecture

We will **derive Direct Preference Optimization (DPO) from scratch**, then look at how it is used in practice.

The promise of DPO:

- No separate reward model
- No reinforcement learning loop
- Just a single, directly-differentiable loss on preference pairs

The plan, in four steps:

1. Solve the RLHF objective for its **optimal policy**
2. Invert that to write the **reward in terms of the policy**
3. Plug it into the **Bradley-Terry** preference model
4. Read off the **DPO loss** and its **gradient**

Rafailov et al., 2023 — "Your Language Model is Secretly a Reward Model" (Rafailov et al., 2023).

Recall: Where DPO sits in the pipeline

Classic RLHF is three moving parts:

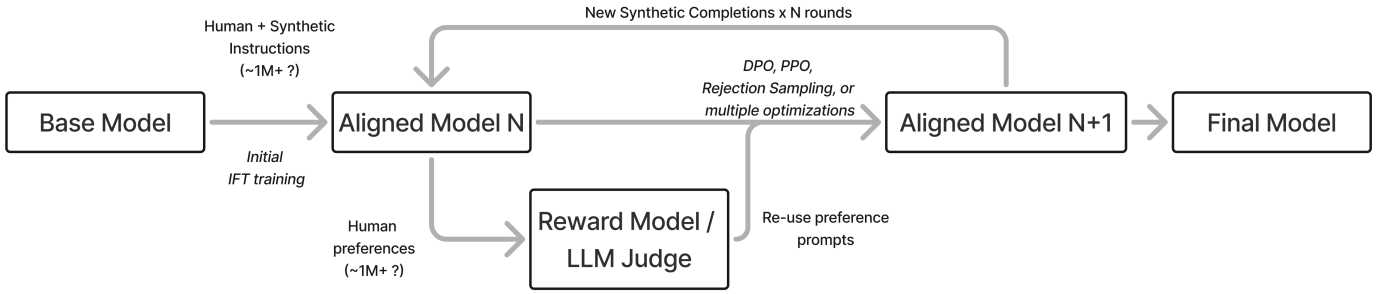
1. Collect human preference pairs
2. Train a reward model $r_\phi(x, y)$
3. Optimize the policy against r_ϕ with RL (e.g. PPO), under a KL penalty

DPO collapses steps 2 and 3 into **one supervised-style loss**.

The key realization we will prove:

> *The optimal RLHF policy and the reward model are two views of the same object. If we know one, we know the other in closed form.*

So we can train the policy *directly* on preferences.



Modern post-training runs many rounds of feedback and optimization; DPO is one of the interchangeable optimizers in that loop.

DPO at a glance: Where we're headed

The whole derivation lands on two objects.

Optimal policy — the RLHF solution in closed form:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\exp\!\left(\tfrac{1}{\beta}r(x,y)\right)$$

DPO loss — what you actually train:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta\log\frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)\right]$$

In practice, that loss is just a few lines:

```
pi_logratios  = policy_chosen_logps  -
policy_rejected_logps
ref_logratios = reference_chosen_logps -
reference_rejected_logps
logits = pi_logratios - ref_logratios
losses = -F.logsigmoid(beta * logits)
```

Everything between here and the implementation is *why* this is the right loss.

DPO derivation P1/4: Deriving the optimal policy

Start with the RLHF optimization problem

$$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi}\Big[\, r(x,y)\,\Big] - \beta\, \mathcal{D}_{\mathrm{KL}}\big(\pi \,\|\, \pi_{\mathrm{ref}}\big) \quad \text{the RLHF objective}$$

We want to find the π that solves this equation! But, without RL.

Fold the KL into the expectation

$$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi}\Big[\, r(x,y) \,\Big] - \beta\, \mathcal{D}_{\mathrm{KL}}\big(\pi \,\|\, \pi_{\mathrm{ref}}\big) \qquad \text{the RLHF objective}$$

$$= \max_{\pi} \; \mathbb{E}\Big[\, r(x,y) - \beta \log \tfrac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)} \,\Big] \qquad \text{write KL} = \mathbb{E}_{x\sim\mathcal{D},y\sim\pi}\Big[\log \tfrac{\pi}{\pi_{\mathrm{ref}}}\Big]$$

Fold the KL into the expectation

$$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi}\Big[\, r(x,y) \,\Big] - \beta\, \mathcal{D}_{\mathrm{KL}}\big(\pi \,\|\, \pi_{\mathrm{ref}}\big) \qquad \text{the RLHF objective}$$

$$= \max_{\pi} \; \mathbb{E}\Big[\, r(x,y) - \beta \log \tfrac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)} \,\Big] \qquad \text{write KL} = \mathbb{E}_{x\sim\mathcal{D},y\sim\pi}\Big[\log \tfrac{\pi}{\pi_{\mathrm{ref}}}\Big]$$

$$= \max_{\pi} \Big(\mathbb{E}[r(x,y)] - \beta\, \mathbb{E}\Big[\log \tfrac{\pi(y|x)}{\pi_{\mathrm{ref}}(y|x)}\Big]\Big) \qquad \text{split into two terms}$$



Flip to a minimization & clean-up

$$\max_{\pi} \left(\mathbb{E}[r(x,y)] - \beta \, \mathbb{E}\Big[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \Big] \right) \quad \text{where we left off}$$

Flip to a minimization & clean-up

$$\max_{\pi} \left(\mathbb{E}[r(x,y)] - \beta \, \mathbb{E}\Big[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \Big] \right) \qquad \text{where we left off}$$

$$= \min_{\pi} \left(-\mathbb{E}[r(x,y)] + \beta \, \mathbb{E}\Big[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \Big] \right) \qquad \text{multiply by } -1 : \ \max \to \min$$

Flip to a minimization & clean-up

$$\max_{\pi} \left(\mathbb{E}[r(x,y)] - \beta \, \mathbb{E}\Big[\log \tfrac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \Big] \right) \qquad \text{where we left off}$$

$$= \min_{\pi} \left(-\mathbb{E}[r(x,y)] + \beta \, \mathbb{E}\Big[\log \tfrac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \Big] \right) \qquad \text{multiply by } -1 : \ \max \to \min$$

$$= \min_{\pi} \mathbb{E}\Big[\log \tfrac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \tfrac{1}{\beta} r(x,y) \Big] \qquad \text{divide by } \beta, \text{ recombine}$$

Fold the optimization target into one log-ratio

Pick up where we left off and fold the bracket into a single log-ratio, using $\frac{1}{\beta} r = \log e^{r/\beta}$:

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) \right] \quad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}} \right]$$

- π is now measured against $\pi_{\text{ref}}\, e^{r/\beta}$: the **reference reweighted by reward** — each response's probability is multiplied by $e^{r/\beta}$, which is larger the higher its reward (β sets how aggressive the reweighting is).
- If that target were a probability distribution q, this $\min_\pi$ would be a **KL divergence**, $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right] = \mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — minimized *exactly*, in closed form (Gibbs' inequality).

Fold the optimization target into one log-ratio

Pick up where we left off and fold the bracket into a single log-ratio, using $\frac{1}{\beta} r = \log e^{r/\beta}$:

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) \right] \quad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}} \right]$$

- π is now measured against $\pi_{\text{ref}}\, e^{r/\beta}$: the **reference reweighted by reward** — each response's probability is multiplied by $e^{r/\beta}$, which is larger the higher its reward (β sets how aggressive the reweighting is).
- If that target were a probability distribution q, this $\min_\pi$ would be a **KL divergence**, $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right] = \mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — minimized *exactly*, in closed form (Gibbs' inequality).

응답	기준확률 π_{ref}	보상 r	e^r	기준확률 $\times e^r$	정규화확률
좋음	0.2	2	7.39	1.478	47.1%
보통	0.5	1	2.72	1.36	43.4%
나쁨	0.3	0	1	0.3	9.5%
				Sum = 3.138	

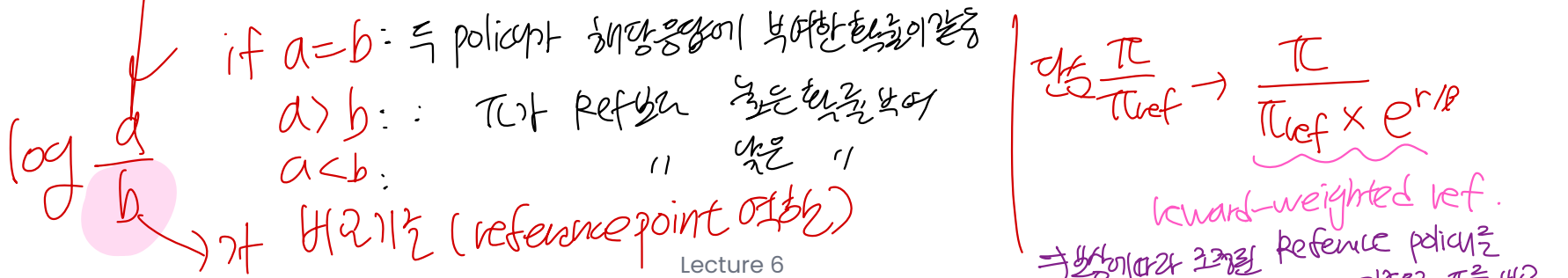
Fold the optimization target into one log-ratio

Pick up where we left off and fold the bracket into a single log-ratio, using $\frac{1}{\beta} r = \log e^{r/\beta}$:

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) \right] \quad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}} \right]$$

- π is now measured against $\pi_{\text{ref}}\, e^{r/\beta}$: the **reference reweighted by reward** — each response's probability is multiplied by $e^{r/\beta}$, which is larger the higher its reward (β sets how aggressive the reweighting is).
- If that target were a probability distribution q, this $\min_\pi$ would be a **KL divergence**, $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right] = \mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — minimized *exactly*, in closed form (Gibbs' inequality).



Fold the optimization target into one log-ratio

Pick up where we left off and fold the bracket into a single log-ratio, using $\frac{1}{\beta} r = \log e^{r/\beta}$:

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) \right] \quad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}} \right]$$

- π is now measured against $\pi_{\text{ref}}\, e^{r/\beta}$: the **reference reweighted by reward** — each response's probability is multiplied by $e^{r/\beta}$, which is larger the higher its reward (β sets how aggressive the reweighting is).
- If that target were a probability distribution q, this $\min_\pi$ would be a **KL divergence**, $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right] = \mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — minimized *exactly*, in closed form (Gibbs' inequality).

Introduce the partition function $Z(x)$

We are minimizing $\min_\pi \; \mathbb{E}_{y\sim\pi}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)\, e^{\, r(x,y)/\beta}} \right].$

Why do we want π to match that denominator?

Introduce the partition function $Z(x)$

We are minimizing $\min_\pi \; \mathbb{E}_{y\sim\pi}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)\, e^{\, r(x,y)/\beta}} \right]$.

Why do we want π to match that denominator? Because $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right]$ is a **KL divergence** $\mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ – it is ≥ 0, and equals 0 **only when** $\pi = q$. So the minimizer is whatever distribution q sits in the denominator.

Introduce the partition function $Z(x)$

We are minimizing $\min_\pi \; \mathbb{E}_{y\sim\pi}\left[\, \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)\, e^{\,r(x,y)/\beta}} \,\right]$.

Why do we want π to match that denominator? Because $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right]$ is a **KL divergence** $\mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — it is ≥ 0, and equals 0 **only when** $\pi = q$. So the minimizer is whatever distribution q sits in the denominator.

The catch: our denominator $\pi_{\text{ref}}\, e^{r/\beta}$ is **not a distribution** — summed over y it does not equal 1.

Introduce the partition function $Z(x)$

We are minimizing $\min_\pi \; \mathbb{E}_{y\sim\pi}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)\, e^{\,r(x,y)/\beta}}\right]$.

Why do we want π to match that denominator? Because $\mathbb{E}_{y\sim\pi}\left[\log \frac{\pi}{q}\right]$ is a **KL divergence** $\mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — it is ≥ 0, and equals 0 **only when** $\pi = q$. So the minimizer is whatever distribution q sits in the denominator.

The catch: our denominator $\pi_{\text{ref}}\, e^{r/\beta}$ is **not a distribution** — summed over y it does not equal 1. We normalize it with the partition function:

$$Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, \exp\left(\tfrac{1}{\beta} r(x,y)\right) \qquad \text{(depends on } x \text{ and } r\text{, not on } \pi\text{)}$$

- Now $q(y \mid x) = \frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x)\, e^{\,r/\beta}$ **is** a valid distribution.
- The objective becomes a genuine $\mathcal{D}_{\text{KL}}(\pi \,\|\, q)$ — minimized exactly at $\pi = q$ (made rigorous with Gibbs' inequality shortly).
- Next we massage the inside of the objective so $Z(x)$ appears.

Revisiting the objective with the partition function $Z(x)$

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y)\right] \quad \text{where we left off}$$

Revisiting the objective with the partition function $Z(x)$

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y)\right] \qquad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) + \log Z(x) - \log Z(x)\right] \qquad \text{add } 0 = \log Z - \log Z$$

Revisiting the objective with the partition function $Z(x)$

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y)\right] \qquad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) + \log Z(x) - \log Z(x)\right] \qquad \text{add } 0 = \log Z - \log Z$$

$$= \min_{\pi} \mathbb{E}\left[\left(\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + \log Z(x)\right) - \log Z(x) - \frac{1}{\beta} r(x,y)\right] \qquad \text{group terms}$$

Revisiting the objective with the partition function $Z(x)$

$$\min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y)\right] \qquad \text{where we left off}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x,y) + \log Z(x) - \log Z(x)\right] \qquad \text{add } 0 = \log Z - \log Z$$

$$= \min_{\pi} \mathbb{E}\left[\left(\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + \log Z(x)\right) - \log Z(x) - \frac{1}{\beta} r(x,y)\right] \qquad \text{group terms}$$

$$= \min_{\pi} \mathbb{E}\left[\log \frac{\pi(y \mid x)}{\frac{1}{Z(x)}\pi_{\text{ref}}(y \mid x)} - \log Z(x) - \frac{1}{\beta} r(x,y)\right] \qquad \log a + \log b = \log ab$$

Turn the objective into a KL divergence

Fold $\frac{1}{\beta} r = \log e^{r/\beta}$ into the denominator of the first term in the objective (again, log rules) and name it $q(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}$ — a valid distribution over y. The objective is:

$$\min_{\pi} \; \mathbb{E}_{x \sim \mathcal{D}} \, \mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y|x)}{q(y|x)} - \log Z(x) \right]$$

$\log Z(x)$ does not depend on y, so pull it out of the inner expectation:

$$\min_{\pi} \; \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim \pi} \left[\log \frac{\pi(y|x)}{q(y|x)} \right] - \log Z(x) \right]$$

The inner expectation is exactly a **KL divergence**, $\mathbb{E}_{y \sim \pi} \left[\log \frac{\pi}{q} \right] = \mathcal{D}_{\text{KL}}(\pi \,\|\, q)$:

$$\min_{\pi} \; \mathbb{E}_{x \sim \mathcal{D}} \left[\mathcal{D}_{\text{KL}}\big(\pi(y \mid x) \,\|\, q(y \mid x)\big) - \log Z(x) \right] \quad (1)$$

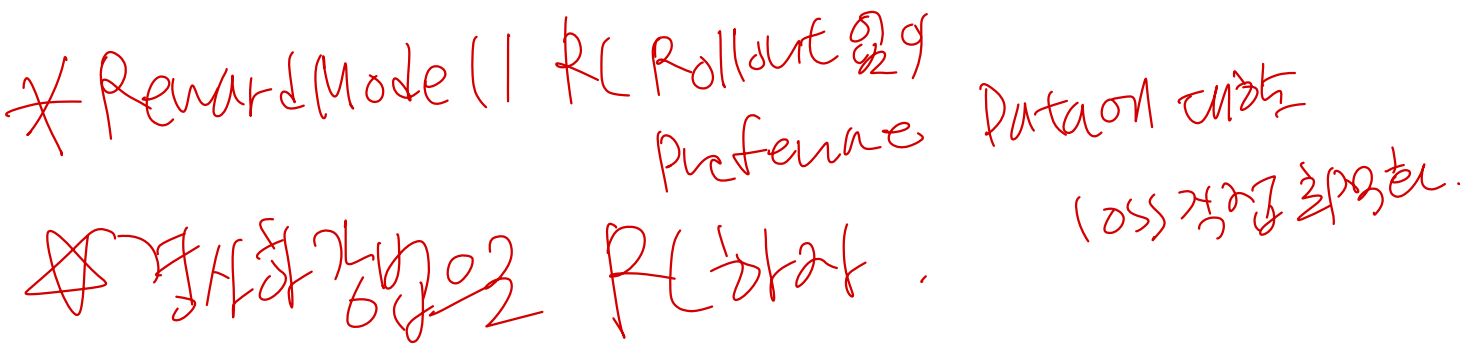
Apply Gibbs' inequality to read off π^*

Where we landed – the objective is now a KL plus a π-independent constant:

$$\min_{\pi} \; \mathbb{E}_{x\sim\mathcal{D}}\Big[\, \mathcal{D}_{\mathrm{KL}}\big(\pi(y \mid x) \,\|\, q(y \mid x)\big) - \log Z(x) \,\Big]$$

- $\log Z(x)$ does not depend on π — only the KL term can be optimized.
- A KL is ≥ 0 and equals 0 **only when** $\pi = q$ (Gibbs' inequality), so the minimizer is $\pi^* = q$:

$$\boxed{\pi^*(y \mid x) = q(y \mid x) = \frac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y \mid x)\, \exp\!\Big(\tfrac{1}{\beta} r(x, y)\Big)}$$



DPO at a glance: Where we're

The whole derivation lands on two objects.

Optimal policy — the RLHF solution in closed form:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\exp\left(\frac{1}{\beta}r(x,y)\right)$$

DPO loss — what you actually train:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta\log\frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)\right]$$

$$\min_\pi \mathbb{E}\left[\log\frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta}r(x,y)\right] \quad \text{where we left}$$

$$= \min_\pi \mathbb{E}\left[\log\frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)\,e^{r(x,y)/\beta}}\right]$$

$D_{KL}(\pi \| q)$

```
pi_
po
ref
ref
log
los
```

Ever

① 출발 RLHF 목적함수

What we want to optimize:

$$\max_{\pi} \mathbb{E}_{x,y}[r(x,y)] - \beta D_{KL}(\pi \| \pi_{ref})$$

reward 최대화 / Reference policy에서 너무 벗어나지 않는다

→ 이 두 조건을 동시에 가장 잘 만족하는 policy는 도대체 어떤 확률분포를 갖나?

→ 수학적으로 정의

② KL항태로 만들기

$$\min_{\pi} \mathbb{E}_{x,y}\left[\log \frac{\pi(y|x)}{\pi_{ref}(y|x)} - \frac{1}{\beta} r(x,y)\right]$$

$$= \min_{\pi} \mathbb{E}_{x,y}\left[\log \frac{\pi(y|x)}{\pi_{ref}(y|x) e^{r(x,y)/\beta}}\right]$$

③ 정규화 상수 $Z(x)$ 정의

$$Z(x) = \sum_{y} \pi_{ref}(y|x) e^{r(x,y)/\beta}$$

$$q(y|x) = \frac{\pi_{ref}(y|x) e^{r(x,y)/\beta}}{Z(x)}$$

확률분포함수

$$\pi_{ref}(y|x) e^{r(x,y)/\beta} = Z(x) \cdot q(y|x)$$

④ $\min_{\pi} \mathbb{E}_{x,y}\left[\log \frac{\pi(y|x)}{Z(x) \cdot q(y|x)}\right]$

$$= \min_{\pi} \mathbb{E}_{x,y}\left[\log \frac{\pi(y|x)}{q(y|x)} - \log Z(x)\right]$$

자가곱정리면 이것상수

$$= \min_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)}\left[\log \frac{\pi(y|x)}{q(y|x)}\right] - \log Z(x)$$

$$\overset{(최종)}{=} \min_{\pi} \mathbb{E}_{x \sim D}\left[\log \frac{\pi(y|x)}{q(y|x)}\right] - \log Z(x)$$

- $D_{KL}(p\|q) = \mathbb{E}_{y \sim p}\left[\log \frac{p(y)}{q(y)}\right]$

⑤ KL적용

$$= \min_{\pi} \mathbb{E}_{x \sim D}\left[D_{KL}(\pi(\cdot|x) \| q(\cdot|x)) - \log Z(x)\right] \quad -(1)$$

$$\pi^*(y|x) = q(y|x) = \frac{\pi_{ref}(y|x) e^{r(x,y)/\beta}}{Z(x)}$$

$$= \frac{1}{Z(x)} \cdot \pi_{ref}(y|x) \cdot \exp\left(\frac{1}{\beta} r(x,y)\right) \quad \square.$$

10시 18분

전체 유도 흐름을 한눈에 보면

$$\min_{\pi} \mathbb{E}_x \mathbb{E}_y \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{r(x,y)}{\beta}\right]$$

$$= \min_{\pi} \mathbb{E}_x \mathbb{E}_y \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x) e^{r(x,y)/\beta}}\right]$$

$$= \min_{\pi} \mathbb{E}_x \mathbb{E}_y \left[\log \frac{\pi(y|x)}{Z(x) q(y|x)}\right]$$

$$= \min_{\pi} \mathbb{E}_x \mathbb{E}_y \left[\log \frac{\pi(y|x)}{q(y|x)} - \log Z(x)\right]$$

$$= \min_{\pi} \mathbb{E}_x \left[\underbrace{\mathbb{E}_y\left[\log \frac{\pi(y|x)}{q(y|x)}\right]}_{D_{\text{KL}}(\pi\|q)} - \log Z(x)\right]$$

$$= \boxed{\min_{\pi} \mathbb{E}_x \left[D_{\text{KL}}(\pi\|q) - \log Z(x)\right]}$$

가장 중요한 부분은 두 가지야.

1. $q = \frac{\pi_{\text{ref}} e^{r/\beta}}{Z}$이므로 분모를 Zq로 치환한다.
2. $\log Z(x)$는 y와 무관하므로 내부 Expectation 밖으로 꺼내고, 남은 항을 KL Divergence로 표현한다.

이 두 단계만 이해하면 전체 유도가 자연스럽게 연결될 거야.

앞) 보상 최대화를 통해 π^*를 유도하는 과정 이해하기.

DPO derivation P2/4: Recovering the reward from the policy (needed to implement it)

Q. 앞에서는 Reward $r(x,y)$를 알고 있을때 최적 policy π^*를 구함.

반대로 최적 policy를 이용해 reward를 표현할 수 있을까?

(intent: DPO: 별도의 RM없이 Policy를 직접 학습하도록 하기 위해서)

(reward models)

Invert π^* for the implicit reward

최적 policy

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)$$

ref policy

최적 policy가 대응하는 Reward

Chap. 1에서 유도한 결과. ⇒

정규화 상수

↓

양변의 log를 취하고 양변정리

↓

Chap. 2활용결과.

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Chap. 1 $r^*(x, y) \rightarrow \pi^*(y|x)$ ←

Chap. 2

목표: Policy가 알맞게 보상을 표현하는 reward를 수학적으로 찾는다.

Invert π^* for the implicit reward

$$\log \pi^*(y \mid x) = \log\left(\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)\right) \quad \text{take } \log \text{ of both sides}$$

Invert π^* for the implicit reward

$$\log \pi^*(y \mid x) = \log\left(\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)\right) \qquad \text{take } \log \text{ of both sides}$$

$$\log \pi^*(y \mid x) = -\log Z(x) + \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r^*(x, y) \qquad \log abc = \log a + \log b + \log c$$

Invert π^* for the implicit reward

$$\log \pi^*(y \mid x) = \log\left(\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)\right) \qquad \text{take } \log \text{ of both sides}$$

$$\log \pi^*(y \mid x) = -\log Z(x) + \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r^*(x, y) \qquad \log abc = \log a + \log b + \log c$$

$$\frac{1}{\beta} r^*(x, y) = \log \pi^*(y \mid x) - \log \pi_{\text{ref}}(y \mid x) + \log Z(x) \qquad \text{rearrange for } r^*$$

Invert π^* for the implicit reward

$$\log \pi^*(y \mid x) = \log\left(\frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x)\, \exp\left(\frac{1}{\beta} r^*(x, y)\right)\right) \qquad \text{take log of both sides}$$

$$\log \pi^*(y \mid x) = -\log Z(x) + \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r^*(x, y) \qquad \log abc = \log a + \log b + \log c$$

$$\frac{1}{\beta} r^*(x, y) = \log \pi^*(y \mid x) - \log \pi_{\text{ref}}(y \mid x) + \log Z(x) \qquad \text{rearrange for } r^*$$

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \qquad \text{multiply by } \beta$$

$$\boxed{r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)}$$

해당응답이 대응하는 "암묵적인" reward

policy의 상대적변화

prompt에만 의존하는항

y 특정응답의 확률을 Ref policy에 비해 최적 policy가 얼마나 높게 책정하는지

모든 prompt x 에대한 모든응답의 공통적을

y_1, y_2의 응답 reward차이 계산

your LM is secretly a reward model

$$r^*(x, y_1) - r^*(x, y_2) = \beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} - \beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)}$$

각 x의 $Z(x)$가 사라짐

⇒ "언제보일지 보여줄지에 Alignment의 특

특정학습의 학습이 얼마나 달라졌는지를 보면,

그 일이 대한 앞으로의 reward를 알 수 있다."

두 행위에 대한 이야기.

DPO = Your language model is secretly a reward model.

RM을 따로 학습시키는게 아니라, LM을 암묵적 보상모델로 본다

DPO derivation P3/4: Connecting to Bradley-Terry preference (what our alignment data has looked like)

⇒ 인간이 두 응답 중 하나를 선호할 확률을 reward 대신 "policy"로 표현할 수 있는가?

chap. 2에서는

$r^*(x,y) \longleftrightarrow \pi^*(x,y)$ 1대1 대응

but, 실제 alignment dataset은 일반적으로 각 응답의 reward 점수를 알려주지 x

(prompt x, 선호된 응답 y_1, 선호되지 않은 응답 y_2)

Recall the Bradley-Terry model

A preference between two responses is a softmax over their rewards:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\big(r^*(x, y_1)\big)}{\exp\big(r^*(x, y_1)\big) + \exp\big(r^*(x, y_2)\big)}$$

Now substitute the implicit reward $r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$.

New here? See Lecture 2 and Chapter 5: Reward Modeling for the Bradley-Terry model.

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Substitute the reward, then cancel $Z(x)$

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)\right)}{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)\right) + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} + \beta \log Z(x)\right)}$$

- Every term carries the same factor $\exp(\beta \log Z(x)) = Z(x)^\beta$.

$$p^*(y_1 \succ y_2 \mid x) = \frac{Z(x)^\beta \cdot \exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)}{Z(x)^\beta \cdot \exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right) + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)}\right) \cdot Z(x)^\beta}$$

$e^{a+b} = e^a \cdot e^b$, $e^{\log a} = a$, $a \log b = \log b^a$

$e^{\beta \log Z(x)} = e^{\log (Z(x))^\beta} = Z(x)^\beta$

- Decompose $e^{a+b} = e^a e^b$ and cancel the shared $Z(x)^\beta$ — the **intractable term is gone**.

Substitute the reward, then cancel $Z(x)$

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)\right)}{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)\right) + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} + \beta \log Z(x)\right)}$$

- Every term carries the same factor $\exp(\beta \log Z(x)) = Z(x)^\beta$.

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)}{\exp\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right) + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)}\right)}$$

$\beta \log \frac{\pi^*(y_i|x)}{\pi_{\text{ref}}(y_i|x)} = \Delta_i$

- Decompose $e^{a+b} = e^a e^b$ and cancel the shared $Z(x)^\beta$ — the **intractable term is gone**.

$= \frac{e^{\Delta_1} \times e^{-\Delta_1}}{(e^{\Delta_1} + e^{\Delta_2}) \times e^{-\Delta_1}} = \frac{1}{1 + e^{\Delta_2 - \Delta_1}}$

$e^a \times e^b = e^{a+b}$

$\Delta_2 - \Delta_1 = -(\Delta_1 - \Delta_2)$ 로 볼수 있음..

※ sigmoid

$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-(\Delta_1 - \Delta_2)}}$

↓

Δ_1, Δ_2를 원래로 되돌리면

Divide through to a sigmoid

Write $\Delta_i = \beta \log \frac{\pi^*(y_i|x)}{\pi_{\text{ref}}(y_i|x)}$, so the cancelled form is $p^* = \frac{e^{\Delta_1}}{e^{\Delta_1} + e^{\Delta_2}}$. Multiply numerator and denominator by $e^{-\Delta_1}$:

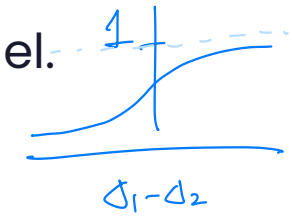
$$p^*(y_1 \succ y_2 \mid x) = \frac{e^{\Delta_1}\, e^{-\Delta_1}}{e^{\Delta_1}\, e^{-\Delta_1} + e^{\Delta_2}\, e^{-\Delta_1}} = \frac{1}{1 + e^{\,\Delta_2 - \Delta_1}}$$

- The numerator becomes 1; the denominator holds an exponential of the **difference** $\Delta_2 - \Delta_1$.

With $\sigma(z) = \frac{1}{1+e^{-z}}$, this is a sigmoid ($\Delta_i = \beta \log \frac{\pi^*(y_i|x)}{\pi_{\text{ref}}(y_i|x)}$):

$$\boxed{p^*(y_1 \succ y_2 \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} - \beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)}\right)}$$

- A **sigmoid of the difference of two log-ratios** — Bradley-Terry, with the policy in place of the reward model.



• $\Delta_1 > \Delta_2$: y_2가 선호될 확률이 0.5보다 작음
• $\Delta_1 = \Delta_2$: 같음
• $\Delta_1 < \Delta_2$: y_1이 선호될 확률이 0.5보다 작음

Bradley-Terry: $\frac{e^{\Delta_1}}{e^{\Delta_1} + e^{\Delta_2}}$ = sigmoid: $\sigma(\Delta_1 - \Delta_2)$

chap 3. Bradley-Terry Model이 implicit reward를 대입하여
선호확률을 sigmoid 형태로 표현. ← chap 2.

DPO derivation P4/4: The loss function and gradient

→ 그 선호확률을 이용해 실제로 최적할 DPO loss를 정의하고, gradient를 유도.

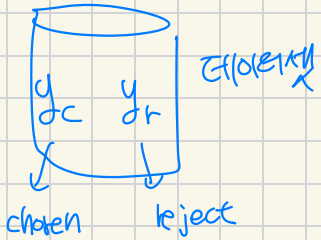
목표 1) 선호확률 $p(y_c > y_r | x)$로 부터 DPO Loss 만들기

목표 2) DPO Loss의 Gradient를 계산해서 모델이 어떻게 학습되는지 이해하기

① 앞에서은은 선호확률

$$p^*(y_1 > y_2 | x) = \sigma\left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{ref}(y_1|x)} - \beta \log \frac{\pi^*(y_2|x)}{\pi_{ref}(y_2|x)}\right)$$

= 두응답 중 y_1이 선호될 확률



what we want : 모델이 y_c을 y_r보다 선호할 확률을 크이자!

선호확률 최대화 : $\max_\theta \mathbb{E}_D[\log p_\theta(y_c > y_r | x)]$

그냥 Loss로 쓰는 것과 부호

$L_{DPO} = -\mathbb{E}_D[\log p_\theta(y_c > y_r | x)]$ ··· NLL

↓ π^*을 학습가능한 π_θ로 바꿔대입.

$$= -\mathbb{E}_D\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\right)\right]$$ ··· DPO Loss

⇒ Reward Model이 없이서 완전 사라짐.

The DPO loss

Minimize the negative log-likelihood of the observed preferences ($y_c \succ y_r$) – this is making the probability more likely:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_c,y_r)\sim\mathcal{D}}\left[\log p(y_c \succ y_r \mid x)\right]$$

Plug in the sigmoid form of p, with the trainable policy π_θ as the implicit reward:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_c,y_r)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)\right]$$

This is directly differentiable — **no reward model, no sampling, no RL loop.**

① 시그모이드의 성질이용

$$= -\mathbb{E}_D\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \beta\log\frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\right)\right] \quad \cdots \text{DPO Loss}$$

$= u$ (시그모이드 내부를 u로 하자)

$$L_{DPO} = -\mathbb{E}_D[\log\sigma(u)]$$ 미분 $\frac{d}{du}\log\sigma(u) = \frac{\sigma'(u)}{\sigma(u)}$, $\sigma'(u) = \sigma(u)(1-\sigma(u))$

what we want: $\nabla_\theta L_{DPO} = -\mathbb{E}_D[(1-\sigma(u))\nabla_\theta u]$ $= \frac{\sigma(u)(1-\sigma(u))}{\sigma(u)} = 1-\sigma(u)$

② u 미분

$$u = \beta\log\frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \beta\log\frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}$$

$$= \beta\left[\log\pi_\theta(y_c|x) - \log\pi_{ref}(y_c|x) - \log\pi_\theta(y_r|x) + \log\pi_{ref}(y_r|x)\right]$$

* π_{ref}는 Frozen Model이기 때문에, 이에 대해 미분하면, 0

$$\nabla_\theta u = \beta\left[\nabla_\theta\log\pi_\theta(y_c|x) - \nabla_\theta\log\pi_\theta(y_r|x)\right]$$

⇒

$$\nabla_\theta L_{DPO} = -\beta\,\mathbb{E}_D[\sigma(-u)\cdot \nabla_\theta\log\pi_\theta(y_c|x) - \nabla_\theta\log\pi_\theta(y_r|x)]$$

* $1-\sigma(u) = \sigma(-u)$

$\sigma(u) = \frac{1}{1+e^{-u}}$, $1-\sigma(u) = 1 - \frac{1}{1+e^{-u}}$

$$= \frac{1+e^{-u}-1}{1+e^{-u}} = \frac{e^{-u}/e^{-u}}{(1+e^{-u})/e^{-u}} = \frac{1}{1+e^{u}} = \sigma(-u)$$

($\frac{1}{e^u+1}$)

③ DPO Gradient 해석하기 - 39p.

$$\nabla_\theta L_{DPO} = -\beta \mathbb{E}_D[\underbrace{\sigma(-u)}_{=w} \cdot \underbrace{\nabla_\theta \log \pi_\theta(y_c|x) - \nabla_\theta \log \pi_\theta(y_r|x)}_{\text{선호응답과 거절응답의 gradient 차이}}]$$

How wrong ...

$\sigma(r_\theta(x,y_r) - r_\theta(x,y_c))$

r_θ는 policy로 부터 정의한 implicit reward

Ref Model에서 얼마나 멀어졌는가 (큰 β는 벗어나지 못하도록)

= chosen 응답의 log-probability를 높이는 방향

Rejected 응답이 // 낮추는 //

if) 모델이 거절된 응답을 더 높이 평가하면

$r_\theta(x,y_r) > r_\theta(x,y_c)$

→ w가 커짐

즉, DPO는 이미 잘 구분하는 데이터보다 잘못 구분하는 데이터에 더 큰 Gradient 가중치 부여

▽ Chap 1 ~ Chap 4 정리

① RLHF objective

$$\max_{\pi} \mathbb{E}[r] - \beta D_{KL}(\pi \| \pi_{ref})$$

② Optimal policy

$$\pi^* = \frac{1}{Z(x)} \pi_{ref} e^{r/\beta}$$

▽ 정리) Reward Model을 별도로 학습하지 않아도 policy 자체의 log-probability로 선호확률을 표현할 수 있다

③ Implicit Reward

$$r^* = \beta \log \frac{\pi^*}{\pi_{ref}} + \beta \log Z(x)$$

④ Bradley-Terry Preference

$$p(y_c > y_r|x) = \sigma(r_c - r_r)$$

*여기서 $Z(x)$ 소거

⑤ 이번 chapter

$$L_{DPO} = -\mathbb{E}[\log \sigma(u)]$$

$$u = \beta \log \frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}$$

The DPO gradient

Start by taking the gradient of the previous loss function.

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\nabla_\theta \, \mathbb{E}_{(x,y_c,y_r)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)\right]$$

Let $u = \left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)$ — the term inside σ.

Chain rule, starting with $\frac{d}{dz}\log\sigma(z) = \frac{\sigma'(z)}{\sigma(z)}$ as the inner operation:

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_c,y_r)}\left[\frac{\sigma'(u)}{\sigma(u)} \nabla_\theta u\right]$$

Substitute $\sigma'(u) = \sigma(u)\big(1-\sigma(u)\big)$ and cancel $\sigma(u)$:

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_c,y_r)}\left[\frac{\sigma(u)\big(1-\sigma(u)\big)}{\sigma(u)} \nabla_\theta u\right] = -\mathbb{E}_{(x,y_c,y_r)}\left[\big(1-\sigma(u)\big) \nabla_\theta u\right]$$

The DPO gradient

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\,\mathbb{E}_{(x,y_c,y_r)}\Big[\big(1-\sigma(u)\big)\,\nabla_\theta u\Big]$$

The reflection identity $1-\sigma(u)=\sigma(-u)$ flips the argument, giving the compact form:

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\,\mathbb{E}_{(x,y_c,y_r)}\Big[\sigma(-u)\,\nabla_\theta u\Big]$$

Now differentiate u, where $-u = \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)} - \beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)}$. Only π_θ depends on θ — π_{ref} is frozen, so its $\log$ terms drop:

$$\nabla_\theta u = \beta\big[\nabla_\theta \log \pi_\theta(y_c \mid x) - \nabla_\theta \log \pi_\theta(y_r \mid x)\big]$$

Substitute $\nabla_\theta u$ and write $\sigma(-u)$ out in full:

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\,\beta\,\mathbb{E}_{(x,y_c,y_r)}\Big[\sigma\big(\beta \log \tfrac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)} - \beta \log \tfrac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)}\big)\,\big[\nabla_\theta \log \pi_\theta(y_c \mid x) - \nabla_\theta \log \pi_\theta(y_r \mid x)\big]\Big]$$

The DPO gradient

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \mathbb{E}_{(x,y_c,y_r)}\Big[\underbrace{w}_{\text{how wrong}} \cdot \big(\nabla_\theta \log \pi_\theta(y_c \mid x) - \nabla_\theta \log \pi_\theta(y_r \mid x)\big)\Big]$$

$$\text{where} \quad w = \sigma\big(r_\theta(x, y_r) - r_\theta(x, y_c)\big)$$

- **Weight** $w \in (0, 1)$ is larger when the model is *more wrong* — when it ranks the rejected response above the chosen the loss is higher.
- **The delta-gradient bracket** raises the likelihood of y_c and lowers that of y_r.
- β scales the step, trading correct ordering against drift from π_{ref} (downstream of the KL penalty in the RLHF objective).

Recap: The DPO derivation

chap 1 – chap 4.

$$\textbf{Objective} \;:\; \max_{\pi} \mathbb{E}\big[r(x,y)\big] - \beta\, \mathcal{D}_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})$$

$$\Rightarrow \textbf{Optimal policy} \;:\; \pi^*(y \mid x) = \tfrac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y \mid x) \exp\big(\tfrac{1}{\beta} r(x,y)\big)$$

$$\Rightarrow \textbf{Implicit reward} \;:\; r^*(x,y) = \beta \log \tfrac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta \log Z(x)$$

$$\Rightarrow \textbf{Bradley-Terry} \;:\; p^*(y_1 \succ y_2) = \sigma\big(r^*(x,y_1) - r^*(x,y_2)\big) \;\; (Z \text{ cancels})$$

$$\Rightarrow \textbf{DPO loss} \;:\; -\log \sigma\big(\beta \log \tfrac{\pi_\theta(y_c)}{\pi_{\mathrm{ref}}(y_c)} - \beta \log \tfrac{\pi_\theta(y_r)}{\pi_{\mathrm{ref}}(y_r)}\big)$$

The reward model never had to be built — it was hiding inside the policy the whole time.

DPO weaknesses, variants, and implementation details

DPO 한계, 변형, 구현세부사항

$L_{DPO} = -\log \sigma(u)$ … DPO Loss

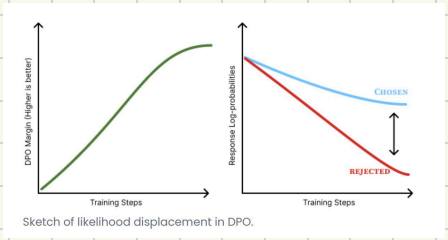
$$u = \beta\left[\log \frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \log \frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\right]$$

DPO는 u를 크게 하려는 항

↳ 하는 방법: chosen의 확률만 높이는게 아닌,
rejected의 확률을 chosen보다 빠르게 낮춰도됨

↳ Likelihood Displacement

↳ 문제: 모델이 chosen에 확률을 더 할당 해야



Sketch of likelihood displacement in DPO.



↳ 을 높여야하는데!

e.g) $\pi_\theta(y_c|x) \geq \pi_{ref}(y_c|x)$

※ DPO는 chosen의 확률이
ref보다 높아야 한다는 제약 ×

DPO는
chosen과 rejected 사이의
상대적 차이에 집중하므로,
두 응답의 확률이
"모두 감소" 하더라도
그 차이가 유지되거나 벌어지면 Loss가 감소

⇒ DPO Loss의 감소가
실제 응답 품질의 향상을 보장하지 않는다.

↳ P) $L = L_{DPO} + \lambda[-\log \pi_\theta(y_c|x)]$

L_{SFT}

chosen의 확률을 직접 높이려는
학습신호 추가

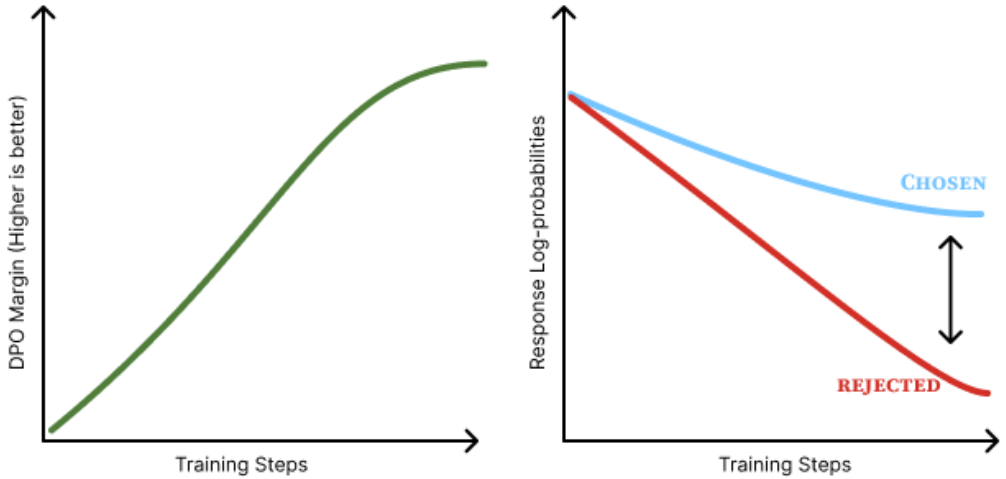
"" 고려해볼, Cal-DPO, AlphaPO

A subtle risk: The chosen probability can fall

The DPO loss only cares about the **margin** between the chosen and rejected log-ratios — not their absolute values. So the model can lower the loss by pushing the *rejected* probability down **faster** than the chosen, even while the **chosen probability also falls**.

Mediated through the partition function $Z(x)$ in the derivation.

- Called **likelihood displacement** (Razin et al., 2025) (Ren & Sutherland, 2025); posited to push probability toward unaddressed, off-distribution behaviors.
- A reason some practitioners add an SFT term on the chosen response, or use fixes like Cal-DPO (Xiao et al., 2024) / AlphaPO (Gupta et al., 2025).



Sketch of likelihood displacement in DPO.

In a real run: Olmo 1B DPO — the chosen/rejected reward margin widens, yet the *chosen* log-prob itself can still drift down.

The β parameter and the static KL

β sets the strength of the KL constraint relative to reward maximization:

- **Large** $\beta \rightarrow$ policy stays close to π_{ref}; it barely moves.
- **Small** $\beta \rightarrow$ policy is free to deviate; it can **over-optimize**.

Crucially, DPO's final KL distance is **static**: it steps directly to the *optimal* solution implied by the dataset and the chosen β. Online RL instead takes steps based on freshly sampled batches and a per-sample KL penalty. Some RL runs even include dynamically adjusted KL controllers.

A zoo of direct alignment algorithms

Each variant tweaks the loss to fix a limitation — often a one-line change. I started calling all the variants Direct Alignment Algorithms (DAAs). Two to start:

1. **Identity Preference Optimization (IPO)** (Azar et al., 2024) — softens the preference probability away from Bradley-Terry to curb overfitting.

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}_{(x,y_w,y_l)}\left[\left(\log \frac{\pi_\theta(y_w|x)\,\pi_{\text{ref}}(y_l|x)}{\pi_\theta(y_l|x)\,\pi_{\text{ref}}(y_w|x)} - \frac{1}{2\beta}\right)^2\right]$$

2. **Conservative DPO (cDPO)** (Rafailov et al., 2023) (in the original DPO paper) — assumes the preference labels are noisy (flipped with probability ε) and softens the loss target accordingly.

3. **Offset DPO (ODPO)** (Amini et al., 2024) — requires the chosen to beat the rejected by a margin offset, scaled by how strong the preference is.

A zoo of direct alignment algorithms

Two more that **drop the reference model** entirely:

- **Odds Ratio Preference Optimization (ORPO)** (Hong et al., 2024) — adds an odds-ratio pull toward the chosen response, folded directly into the SFT loss. The odds ratio uses only π_θ, which allows dropping the reference model.

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} - \lambda \mathbb{E}\left[\log \sigma\left(\log \frac{\text{odds}_\theta(y_w|x)}{\text{odds}_\theta(y_l|x)}\right)\right], \quad \text{odds}_\theta(y) = \frac{\pi_\theta(y)}{1-\pi_\theta(y)}$$

- **Simple Preference Optimization (SimPO)** (Meng et al., 2025) — length-normalizes the reward into an average log-prob, with a target margin γ. $\frac{1}{|y|}$ normalizes by length; γ is a target margin; no π_{ref}.

$$\mathcal{L}_{\text{SimPO}} = -\mathbb{E}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l \mid x) - \gamma\right)\right]$$

The algorithm matters **far less** than the base model and the data. Still, many papers continued to make minor algorithmic tweaks. Many more exist than were on these slides.

Implementation is genuinely simple

No generation during training, no separate reward model — the heart of the loss is a few lines:

```
# log-prob gaps for policy and frozen reference
pi_logratios  = policy_chosen_logps - policy_rejected_logps
ref_logratios = reference_chosen_logps - reference_rejected_logps

# positive when policy shifts mass toward the chosen completion
logits = pi_logratios - ref_logratios
losses = -F.logsigmoid(beta * logits)
```

ref_chosen_logps ($\log \pi_{ref}(y_c|x)$)
ref_reject_logps ($\log \pi_{ref}(y_r|x)$)

Tip: π_{ref} is frozen, so precompute and cache its log-probs to cut peak memory ~50%. Reference code: `code/direct_alignment/`.

⇒ peak memory 약 50% 절약 ㅇ

$$L_{DPO} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)} - \log \frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\right]\right)$$

$$u = \beta\left[\log \pi_\theta(y_c|x) - \log \pi_\theta(y_r|x) - \left(\log \pi_{ref}(y_c|x) - \log \pi_{ref}(y_r|x)\right]\right.$$

π_θ − logratios

ref_logratios

DAAs work with synthetic preference data

These algorithms need *feedback* data, not necessarily *human* feedback data — AI feedback works just as well.

- Most modern DPO uses preferences labeled by a strong model. **UltraFeedback** (Cui et al., 2023) was the first prominent *open* synthetic preference dataset and helped kickstart the DPO era; Tülu 3 (Lambert et al., 2024), SmolLM 2 (Ben Allal et al., 2025), and others followed with larger synthetic-feedback recipes.
- **On-policy data** (some completions from the model you are tuning) helps the contrastive loss optimize the right token space within a complex post-training recipe (studied this specifically in Tülu 3).
- **Delta Learning** (Geng et al., 2025): later work argues the *gap* between chosen and rejected matters more than which models produced them (e.g. Qwen3-32B chosen vs Qwen3-0.6B rejected). Used this in Olmo 3!

On-policy data: 현재 학습하는 모델이 직접 생성한 응답을 preference 데이터에 포함하는 거야. 이렇게 하면 실제 모델이 생성하는 응답 영역에서 선호도를 학습할 수 있어.

Delta Learning: Chosen과 rejected가 어떤 모델에서 생성되었는지보다, 두 응답 사이의 품질 차이가 중요할 수 있다는 관점이야.

예를 들어 Qwen3-32B의 좋은 응답과 Qwen3-0.6B의 나쁜 응답을 하나의 preference pair로 구성하는 방식이지.

48p. DPO vs. RL: Offline vs. Online

이 페이지는 DPO와 PPO의 가장 중요한 차이를 비교해.

	DPO	PPO / Online RL
데이터	미리 수집된 고정 데이터	학습 중 새롭게 생성
Reward Model	별도로 필요 없음	일반적으로 사용
학습 방식	Preference Loss 최적화	Reward 기반 Policy Gradient
계산 비용	상대적으로 낮음	상대적으로 높음
새로운 응답 탐색	제한적	가능
성능 상한	데이터 coverage에 제한	더 높은 성능 가능성

예를 들어 어떤 모델이 수학 문제를 푸는 능력을 학습한다고 해보자.

DPO는 미리 만들어 둔 좋은 풀이와 나쁜 풀이를 비교하면서 학습해.

반면 PPO는 현재 모델이 직접 문제를 풀어보고, 그 결과에 대한 reward를 받아 policy를 업데이트할 수 있어.

따라서 Online RL은 학습 중 새로운 전략을 발견할 가능성이 있어.

하지만 그만큼 generation, reward 평가, policy update 등 복잡한 과정이 필요해.

DPO는 단순하고 효율적이지만 데이터가 다루지 않은 행동을 새롭게 탐색하기 어렵다는 것이 핵심적인 한계야.

DPO vs. RL: Offline vs. online

DPO and other DAAs

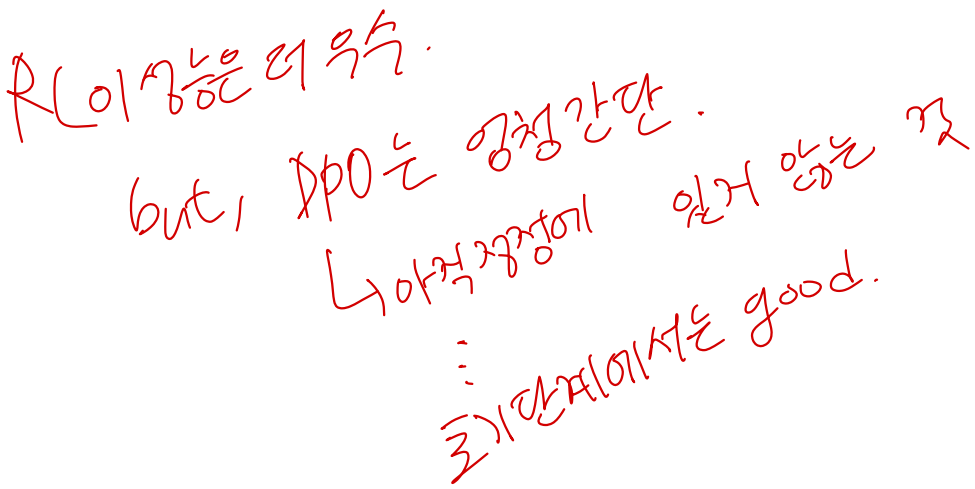
- Train on a fixed dataset collected ahead of time.
- Simpler, more stable, fast to iterate on data.
- Limited by the **coverage** of that dataset — a slightly lower performance ceiling.

PPO / policy gradient is online

- Generate fresh completions during training, score with a reward model.
- Can explore new regions → often higher peak performance (Ivison et al., 2024) (Xu et al., 2024) (Tajwar et al., 2024).
- More compute, more moving parts (four models in memory), more engineering complexity.

Where does this leave us today?

If so many models have used DPO well and it's so simple, why does it seem like it comes up so infrequently – especially at the frontier?



Hypotheses for DPO's role today

- **Olmo 3** (Nov 2025) / **SmolLM 3** (Jul 2025) – used DPO over reasoning traces after SFT to boost performance in a simple pipeline.
- **NVIDIA Nemotron 3** (Dec 2025) – *Mixed Preference Optimization*: DPO + Binary Classifier Optimization in one offline stage, over data scored by a generative reward model.
- **Liquid AI LFM2** (Nov 2025) – length-normalized DPO on semi-online data, before a final RLVR pass.

Hypotheses for DPO's role today

- **Olmo 3** (Nov 2025) / **SmolLM 3** (Jul 2025) – used DPO over reasoning traces after SFT to boost performance in a simple pipeline.
- **NVIDIA Nemotron 3** (Dec 2025) – *Mixed Preference Optimization*: DPO + Binary Classifier Optimization in one offline stage, over data scored by a generative reward model.
- **Liquid AI LFM2** (Nov 2025) – length-normalized DPO on semi-online data, before a final RLVR pass.

How I see things:

- DPO works well in wonky, distillation-heavy recipes like Olmo – e.g. a model with a spikier distribution, with training data from many teacher models.
- DPO still works in other settings, but most labs have the engineering resources to do other things with higher peak performance.
- TLDR: DPO is a path to a good/solid model, but not to the best model. And DPO may do less in *cleaner* training recipes.

References (1/2)

Amini, A., Vieira, T., and Cotterell, R.. "*Direct preference optimization with an offset*." *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Azar, M., Guo, Z., Piot, B., Munos, R., Rowland, M., et al.. "*A general theoretical paradigm to understand learning from human preferences*." *International Conference on Artificial Intelligence and Statistics*, 2024.

Ben Allal, L., Lozhkov, A., Bakouch, E., Blázquez, G., Penedo, G., et al.. "*SmolLM2: When Smol Goes Big -- Data-Centric Training of a Small Language Model*." 2025. [link]

Cui, G., Yuan, L., Ding, N., Yao, G., Zhu, W., et al.. "*Ultrafeedback: Boosting language models with high-quality feedback*." 2023.

Geng, S., Ivison, H., Li, C., Sap, M., Li, J., et al.. "*The Delta Learning Hypothesis: Preference Tuning on Weak Data can Yield Strong Gains*." *Second Conference on Language Modeling*, 2025. [link]

Gupta, A., Tang, S., Song, Q., Zhu, S., Hong, J., et al.. "*AlphaPO--Reward shape matters for LLM alignment*." *International Conference on Machine Learning (ICML)*, 2025.

Hong, J., Lee, N., and Thorne, J.. "*Reference-free monolithic preference optimization with odds ratio*." *arXiv e-prints*, 2024.

Ivison, H., Wang, Y., Liu, J., Pyatkin, V., Lambert, N., et al.. "*Unpacking DPO and PPO: Disentangling Best Practices for Learning from Preferences*." *arXiv preprint arXiv:2406.09279*, 2024.

References (2/2)

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., et al.. "*Tülu 3: Pushing Frontiers in Open Language Model Post-Training*." *arXiv preprint arXiv:2411.15124*, 2024.

Meng, Y., Xia, M., and Chen, D.. "*Simpo: Simple preference optimization with a reference-free reward*." *Advances in Neural Information Processing Systems*, 2025.

Rafailov, R., Sharma, A., Mitchell, E., Manning, C., Ermon, S., et al.. "*Direct preference optimization: Your language model is secretly a reward model*." *Advances in Neural Information Processing Systems*, 2023.

Razin, N., Malladi, S., Bhaskar, A., Chen, D., Arora, S., et al.. "*Unintentional unalignment: Likelihood displacement in direct preference optimization*." *International Conference on Learning Representations (ICLR)*, 2025.

Ren, Y., and Sutherland, D.. "*Learning dynamics of llm finetuning*." *International Conference on Learning Representations (ICLR)*, 2025.

Tajwar, F., Singh, A., Sharma, A., Rafailov, R., Schneider, J., et al.. "*Preference fine-tuning of llms should leverage suboptimal, on-policy data*." *International Conference on Machine Learning (ICML)*, 2024.

Xiao, T., Yuan, Y., Zhu, H., Li, M., and Honavar, V.. "*Cal-dpo: Calibrated direct preference optimization for language model alignment*." *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Xu, S., Fu, W., Gao, J., Ye, W., Liu, W., et al.. "*Is dpo superior to ppo for llm alignment? a comprehensive study*." *International Conference on Machine Learning (ICML)*, 2024.